to underscore (_).

```
>>> num1, _, num2 = [ 1,2,3 ]
>>> print ( num1 )
1
>>> print ( num2 )
3
```

The '*' symbol is used as a prefix to underscore (_) to unpack multiple values and assign them to a list.

```
>>> num1, *_, numlist2 = [ 1,2,3,4,5,6 ]
>>> print ( num1 )
1
>>> print ( numlist2 )
6
```

Getting a password from a user

An easy way to get a password as input from the user is to use the *getpass* function defined in the *getpass* module. The function displays a prompt and then prevents echo on the display when the user types the password.

```
!pip install getpass

from getpass import getpass
uname = input ( "Username: " )
password = getpass ( " Password: " )
```

List comprehension

List comprehension offers an easy way to create a new list in a single line, compared to the traditional methods. An example is given below. If we want to create a new list using the *for* loop, the code will be as follows:

```
#create a list containing the cube of the first 9 integers
>>> cubelist = [ ]
>>> for i in range ( 1,10 ):
>>>     cubelist.append ( i * i  * I )
>>> print ( cubelist )
[ 1, 8, 27, 64, 125, 216, 343, 512, 729 ]
```

The same can be written using list comprehension in the following way in a single line of code:

```
>>> cubelist = [ i * i * i  for i in range (1,10) ]
>>> print ( cubelist )
```

You can even use it for conditional logic and filtering.

```
>>> numlist1 = [ 35, 49, 45, 75, 90, 15, 65, 63, 84, 54 ]
>>> #create a new list whose elements are divisible by 7
>>> numlist2 = [ i for i in numlist1 if i % 7 == 0 ]
>>> print ( numlist2 )
[ 35, 49, 63, 84 ]
```

zip() function

The *zip()* function is an inbuilt function in Python that takes two or more iterable contents passed as arguments like list, dictionary, string or even iterables that are user defined, and combines them to create a single tuple, an iterator object. The *zip* function combines the elements with the same index in pairs (or more if the number of iterators is more) in a tuple. The first item of each argument iterator is combined, then the second is combined, and so on, until the length of the shorter iterable is covered, by default, to create a tuple object. Given below is a simple example:

```
>>> names = [ "Ganesh","Siva Sundar","Gunasekhar","Santosh" ]
>>> marks = [ 90, 88, 75, 72 ]
>>> marklist = tuple ( zip ( names, marks ) )
>>> print ( marklist )
(('Ganesh', 90), ('Siva Sundar', 88), ('Gunasekhar', 75),
('Santosh', 72))
```

The *zip* function is extremely powerful as it can take iterators like strings and dictionaries as arguments. It can also take more than two iterables as arguments, and the iterables can be of varying lengths. It even gives options for iterating all the elements of the longer tuple if the iterable arguments are not of the same length.

Combining two lists into a dictionary

We can combine two lists and create a dictionary by using the *zip* function.

```
>>> names = [ "Ganesh", "Siva Sundar", "Gunasekhar",
"Santosh" ]
>>> marks = [ 90, 88, 75, 72 ]
>>> marklist = dict ( zip ( names, marks ) )
>>> print ( marklist )
{'Ganesh': 90, 'Siva Sundar': 88, 'Gunasekhar': 75,
'Santosh': 72}
```

The *pass* keyword

The *pass* keyword can be used as a placeholder for incomplete code. This keyword is a statement by itself and results in NOP (no operations) as it is discarded during the byte-compile phase and nothing is executed. It is useful for code that you want to add later. Two examples are given below.

Example 1:

```
def passDemo ():
```